

RAGE: Defensa Multi-Turno contra Prompt Injection en Agentes Text-to-SQL

Authors: [Nombre Apellido]¹, [Nombre Apellido]² — [Afiliación / Hub AI Safety México]

Track: AI Security · Sub-track: Prompt injection & jailbreaks

Code and Data: <https://github.com/LuisUPY/RAGE-HACKATHON-PROYECT> · Benchmark holdout: [rage_core/kb/eval_generalization/](#)

Abstract

Agentic Text-to-SQL systems expose a critical attack surface: adversaries can migrate a legitimate session toward destructive queries or data exfiltration over multiple turns without triggering stateless filters. Russinovich et al. (2024) showed that Crescendo jailbreaks achieve 98–100% success on aligned models using only seemingly benign prompts distributed across N turns—the attack is the trajectory, not any single message. We present RAGE (Retrieval-Augmented Governance Engine), a four-layer security gateway for tool-connected LLM agents. Layer 1 applies deterministic signatures; Layer 2 scores similarity against a curated OWASP threat knowledge base; Layer 3 computes session-aware semantic drift (turn-to-turn δ and cumulative Δ from baseline T_0) and invokes an LLM judge only on suspicious turns; Layer 4 fuses signals into a score and action band. A deterministic SQL action gateway blocks destructive operations regardless of upstream decisions. We introduce AUC-D (Area Under the Degradation Curve) and TRI (Temporal Resistance Index)—non-circular metrics grounded in observable compromise (canary leak, prohibited SQL execution). On a 60-case generalization holdout (texts not in the training KB): 80.6% attack recall, 100% precision, 0% benign false-positive rate; the flagship Crescendo scenario is blocked before exfiltration turns T_4 – T_5 . Main takeaway: session-aware defenses are necessary for agentic systems; RAGE demonstrates a reproducible, low-cost pipeline (offline L1+L2, optional judge) suitable for Global South deployments with limited API budgets.

1. Introduction

Organizations increasingly deploy conversational agents that translate natural language into SQL over operational databases (sales, CRM, payroll). Unlike rigid REST APIs, the conversational channel accepts arbitrary, multi-turn input—creating incompatibility with least-privilege database security.

Threat model. We assume an adversary who (a) interacts over multiple turns, (b) may embed injections in indirect content (tickets, documents), and (c) aims to exfiltrate PII, leak system prompts/canaries, or execute destructive SQL via agent tools. We do not assume white-box model access.

Failure mode. Stateless defenses scan each turn in isolation for signatures such as `DROP TABLE` or “ignore previous instructions.” Crescendo-style attacks keep per-turn drift below threshold while cumulative intent migrates far from the benign baseline established at T_0 .

Contributions (new work during the hackathon):

1. Dynamic Semantic Filter with State (Layer 3): cumulative drift Δ alongside turn-to-turn δ , with OWASP LLM08 sanitization of stored intent summaries.

- 2. Temporal metrics AUC-D and TRI: ground-truth vulnerability scoring over conversation time, avoiding circular evaluation.
- 3. Connected agent defense: four-layer cascade plus SQL action gateway and Training-Center red-teaming loop for KB hardening.

2. Related Work

Crescendo [1] formalizes multi-turn foot-in-the-door jailbreaking; we directly target its trajectory-based failure mode. Single-turn jailbreaks [5, 6] motivate Layer 1 but do not address session migration. OWASP LLM Top 10 [4] (LLM01 Prompt Injection, LLM06 Excessive Agency, LLM08 Untrusted Input) maps to our layer design. JailbreakBench [8] focuses on single-turn robustness benchmarks; we complement with multi-turn holdout evaluation outside the threat KB.

Gap addressed: existing guardrails lack (i) baseline-anchored cumulative drift, (ii) action-level containment for tool-using agents, and (iii) non-circular temporal metrics for multi-turn resilience.

3. Methods

Architecture. User turn → L1 (regex) → L2 (RAG/KB cosine similarity) → L3 (embedding drift + optional LLM judge) → L4 (score fusion, optional session EWMA ratchet) → Action Gateway → SQLite agent → AUC-D/TRI evaluator.

Table 1 — Layer summary

Layer	Role	Default cost
L1	14 compiled rules (override, DAN, DROP, [SYSTEM], ...)	O(1), no API
L2	~70 OWASP patterns in threats.json; hot-update supported	TF-IDF offline
L3	Δ vs previous turn; judge if suspicious and L1/L2 unconfirmed	Embedding + optional API
L4	Score 0-100 → ALLOW / WARN / BLOCK	Local
Gateway	Allowlisted tables; blocks DROP, GRANT, UNION, TRUNCATE	Deterministic

Access policy. Block on confirmed injection (L1 or tiered L2 match), LLM judge flag, or multi-turn contextual escalation (is_multiturn_attack_verdict using prior L2/drift peaks and session risk).

Evaluation setup.

- Automated regression: 206 pytest tests (./scripts/run-tests.sh) verify code contracts (gateway, layers, AUC, datasets). Passing pytest ≠ 100% attack recall.
- Open-world security: ./scripts/run-bench-generalization.sh on 60-case holdout outside the training KB; CI enforces ~80% recall and 0% FP (test_generalization_combined_recall_band), not a curated 100%.
- Demo: 33 scenarios via rage-demo with optional LLM judge.
- Reproduce: uv sync → ./scripts/run-tests.sh -q → ./scripts/run-bench-generalization.sh

Design choices not fully successful. Pure regex (L1-only) misses subtle social engineering (~20% FN). TF-IDF embeddings trade quality for zero-download deployment; sentence-transformers optional. Demo agent responses are simulated—the defense pipeline is real; end-to-end ASR against commercial LLMs is left for future work.

4. Results

Table 2 — Generalization holdout (L1+L2, no judge)

Metric Value
Total cases 60
Attack recall 80.6% (29/36)
Precision 100%
Benign FP rate 0% (0/24)
Multi-turn scenarios OK 10/12
Detection by layer (TP) L1: 28 · L2: 1
Runtime ~0.8 s

False negatives (7). Subtle social engineering (“board deck” receivables), credential fishing (Bearer token “smoke test”), PII exfiltration with technical framing, and low-drift Crescendo variants—no trigger keywords, low L2 similarity. LLM judge path (--full) targets these borderline cases.

Figure 1 (conceptual). Crescendo escalation (crescendo_escalation, 6 turns): benign sales queries (T0-T1) → scope expansion (T2-T3, WARN/ratchet) → UNION ALL exfiltration to system_config (T4-T5, BLOCK via cumulative Δ, L4, and gateway \bUNION\b). Undefended baseline reaches compromise at later turns; defended AUC-D ≪ baseline (hypothesis H1, validated in tests).

Attack families covered. Direct override, jailbreak/DAN, indirect injection, payload splitting, Crescendo/salami, many-shot, executive/compliance fraud, PII exfiltration, SQL/tool abuse, prompt/canary leak, multilingual (FR).

5. Discussion and Limitations

Implications. RAGE shows that session-aware monitoring is necessary for agentic Text-to-SQL in sensitive data environments—a relevant problem for Latin American enterprises adopting internal copilots faster than security maturity.

Limitations. (1) TF-IDF default embedder is less semantically dense than transformers. (2) ~20% FN on holdout without judge. (3) Domain scoped to sales/CRM SQL demo. (4) Gateway regex may miss novel SQL obfuscation. (5) Legitimate long multi-topic conversations may elevate drift.

Dual-use risks (mandatory). Publishing drift thresholds, EWMA parameters, and TRI enables adversaries to optimize evasive Crescendo trajectories. The Training-Center can serve as an offline adversarial testbed. Mitigations: per-tenant threshold calibration, rate limiting, keep gateway as deterministic last line, restrict Training-Center to isolated CI.

Future work. Ablation table (L1 vs L1+L2 vs L3 vs full), commercial LLM ASR experiments, SDK integration, many-shot long-context defenses [7].

6. Conclusion

Multi-turn prompt injection against tool-using agents cannot be solved with stateless keyword filters. RAGE combines baseline-anchored semantic drift, RAG-governed threat memory, optional LLM judging, and deterministic action gating into a reproducible pipeline achieving 80.6% recall with zero benign false positives on an out-of-KB holdout. Temporal metrics AUC-D and TRI quantify when defenses fail, not just whether a single turn was flagged. For the Global South context, the offline-first design (L1+L2 without

API keys) lowers the barrier to deploying meaningful agent security. With additional validation against live LLMs and published ablations, RAGE is a credible foundation for agentic AI security research and practical pilots.

References

- [1] Russinovich, M., Salem, A., & Eldan, R. (2024). Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. arXiv:2404.01833. <https://arxiv.org/abs/2404.01833>
- [2] Zhao, W. X., et al. (2023). A Survey of Large Language Models. arXiv:2303.18223.
- [3] Deng, R., et al. (2023). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. arXiv:2308.15363.
- [4] OWASP Foundation (2023). OWASP Top 10 for Large Language Model Applications. LLM01, LLM06, LLM08.
- [5] Zou, A., et al. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043.
- [6] Perez, P., & Ribeiro, S. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527.
- [7] Anil, R., et al. (2024). Many-Shot Jailbreaking. Anthropic Technical Report.
- [8] Chao, P., et al. (2024). JAILBREAKBENCH: An Open Robustness Benchmark. arXiv:2404.01318.

LLM Usage Statement:

LLMs (Cursor IDE, optional NVIDIA/OpenAI judge API) assisted code development, scenario drafting, and manuscript editing. All quantitative claims (recall, precision, test counts, runtime) were verified by running `pytest` and `./scripts/run-bench-generalization.sh` on the repository. The authors take full responsibility for the reported results.

Template: Based on
[\[aisafetymexico/global-south-ais-template\]](https://github.com/aisafetymexico/global-south-ais-template)(<https://github.com/aisafetymexico/global-south-ais-template>).